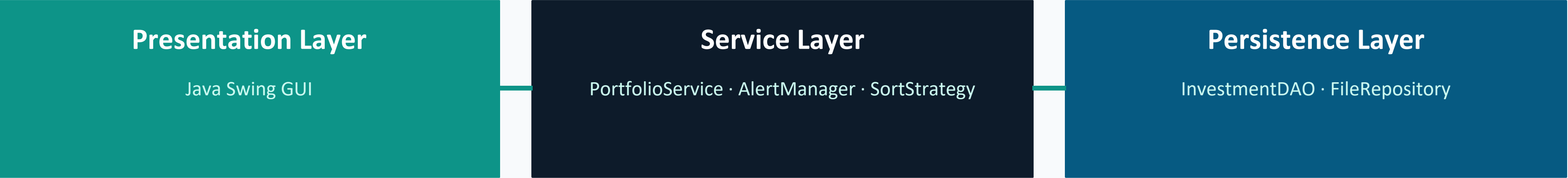


Personal Investment Portfolio Tracker

Investors juggle stocks, mutual funds, and bonds across disconnected tools missing real-time return calculations and threshold alerts.

Target: Retail investors & finance students



Domain Model



Package Structure

model/	Investment, Stock, MutualFund, Bond
service/	PortfolioService, SortStrategy, AlertListener, MonitorThread
repository/	InvestmentDAO, FileInvestmentRepository
exception/	InvalidInvestmentException, PortfolioPersistenceException
ui/	PortfolioDashboard, AddDialog, EditDialog
util/	IdGenerator

OOP Concepts Used

A

Abstraction

Abstract Investment class defines `calculateReturn()` & `getRiskLevel()` without implementation

I

Inheritance

Stock, MutualFund, Bond extend Investment and inherit common attributes

P

Polymorphism

PortfolioService calls `calculateReturn()` on `List<Investment>` — each type responds differently

E

Exceptions

`InvalidInvestmentException`
`PortfolioPersistenceException`
caught at service boundary

Observer

AlertListener interface —
PortfolioDashboard registers
for threshold breach alerts

↕

Strategy

SortStrategy interface —
`SortByReturn` / `Value` / `Risk`
swapped at runtime

Technical Highlight

Polymorphic Return Calculation

```
// Stock – market price based
public double calculateReturn() {
    return currentMarketPrice * qty
        - purchasePrice * qty;
}

// Bond – coupon rate × time held
public double calculateReturn() {
    double years = daysHeld / 365.0;
    return invested * couponRate * years;
}
```

Why: Each asset computes return differently by nature. Using polymorphism eliminates branching in PortfolioService — new asset types can be added without modifying existing logic (Open/Closed principle).

DAO Persistence Pattern

```
// Interface – decouples service from storage
public interface InvestmentDAO {
    void
    save(List<Investment> list);
    List<Investment>
    load();
}

// Concrete: serialized .dat file
class FileInvestmentRepository
    implements InvestmentDAO { ... }
```

Why: Swapping storage from file to database only requires a new DAO implementation — PortfolioService never changes. Isolates I/O failure in a single class with a dedicated exception.

Live Demo

01 Meaningful Scenario

- Add a Stock (Reliance, ₹2400 → ₹2750)
- Add a MutualFund (Axis Bluechip, NAV ₹42 → ₹51)
- Add a Bond (Govt 8%, 3 yrs held)
- Show portfolio summary — total invested vs current value
- Sort by return % — observe polymorphic values differ

02 Threshold Alert

- Set gain alert threshold to 15%
- Update Stock market price to trigger breach
- Observer fires — alert banner updates in real-time
- Background MonitorThread continuously re-evaluates

03 Error Handling Case

- Try adding investment with negative purchase price
- InvalidInvestmentException caught — error dialog shown
- Try entering letters in numeric field — NumberFormatException
- Close app, reopen — portfolio reloads from .dat file (DAO)

```
$ javac -d out $(find src -name '*.java') && java -cp out Main
```